Лабораторная работа № 4. Растеризация окружностей

Цель работы – изучение алгоритмов растеризации окружностей, создание программы для визуализации работы алгоритмов.

Краткие теоретические сведения

Существует несколько простых, но неэффективных способов преобразования окружностей в растровую форму.

Рассмотрим окружность с центром в начале координат, для которой $x^2+y^2=R^2$.

Решая это уравнение относительно y, получим $y = \pm\sqrt{R^2 - x^2}$.

Чтобы изобразить четвертую часть окружности, можно увеличивать x с единичным шагом от 0 до R и на каждом шаге вычислять y и делать округление полученного значения (остальные четверти изображаются симметрично). Этот метод работоспособен, но не эффективен, поскольку в него входят операции умножения и извлечения квадратного корня. Более того, при значениях x, близких к R, в окружности появляются заметные незаполненные промежутки, так как при этих значениях x тангенс угла наклона касательной к окружности стремится к бесконечности (рис. 1).

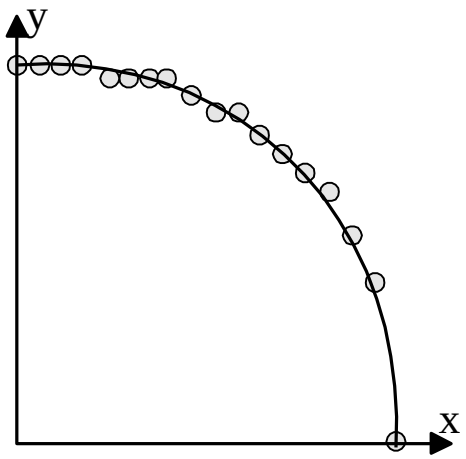


Рис.1. Четверть окружности, построенная с использованием единичных шагов по x и вычисленная округлённых значений y

Ещё один простой подход к построению окружности также заключается в использовании её параметрического уравнения:

$$\begin{cases} x = x_0 + R cos\tau, \\ y = y_0 + R sin\tau, \end{cases}$$

где x_0, y_0 – координаты центра;

R - радиус окружности;

τ - параметр.

При этом, учитывая тот факт, что шаг изменения аргумента должен составлять величину τ=1/R, потребуется рассчитать для каждого значения параметра τ значения координат соответствующих точек окружности и соединить их затем отрезками прямых. Такой путь также требует довольно большого количества вычислений.

Другим, более простым и эффективным путём является использование алгоритма Брезенхема для генерации окружности. Чтобы построить полную окружность, достаточно сгенерировать её одну восьмую часть. Остальные части получаются затем симметричным отражением относительно определённой прямой. Так, отражая одну восьмую часть, построенную в первом октанте для углов в диапазоне 0-45°, относительно прямой с уравнением $y=x$, получим одну четвертую часть, лежащую в первом квадранте. Отразив эту четверть относительно прямой $x=0$, получим одну вторую часть, лежащую выше оси абсцисс, наконец, отразив эту полуокружность относительно прямой $y=0$, получим полную окружность (рис. 2).

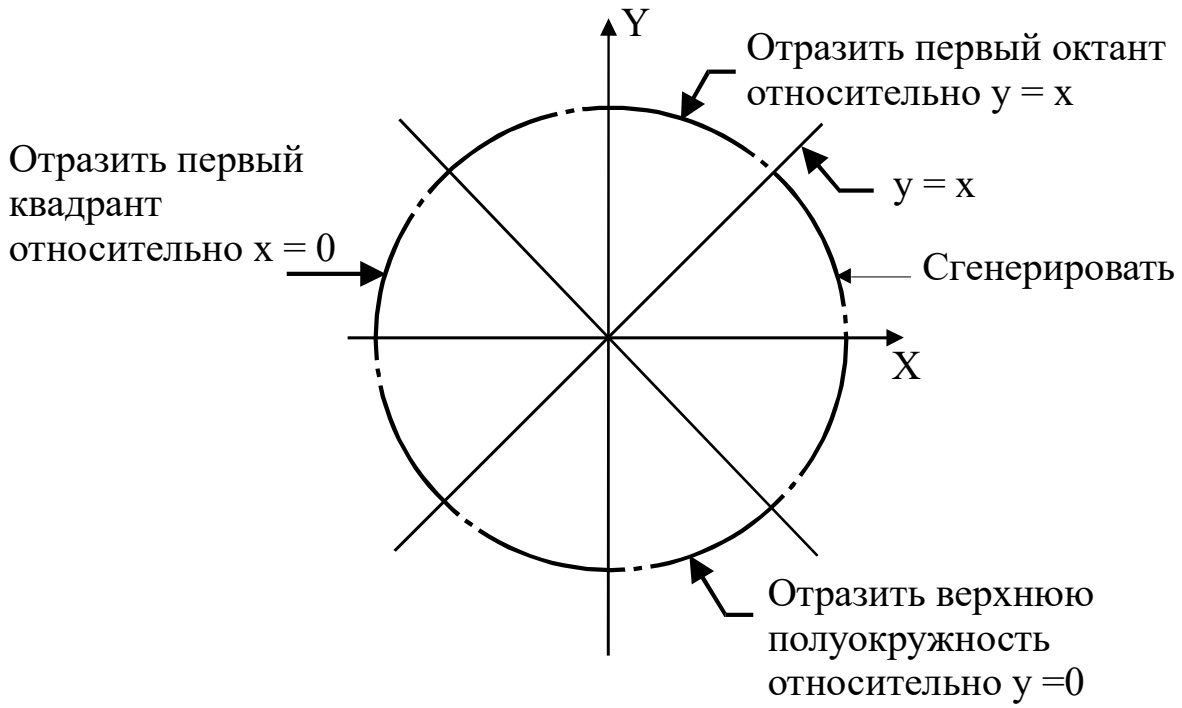


Рис. 2. Генерация полной окружности из дуги в первом октанте

Рассмотрим построение части окружности, лежащей в первом квадранте. Предположим, что центр окружности лежит в начале координат, её радиус равен R, ось абсцисс направлена вправо, ось ординат — вверх. Пусть алгоритм начинает работу в точке с координатами $x=0$, $y=R$, тогда он заканчивает работу в точке $x=R$, $y=0$. При таком направлении построения окружности y как функция аргумента x будет являться монотонно убывающей ($y = \sqrt{R^2 - x^2}$).

Для любого пикселя при выбранном направлении генерации окружности существует только три варианта выбора следующего пикселя, наилучшим образом аппроксимирующего окружность:

– горизонтальный пиксель вправо;
– диагональный вниз и вправо;
– вертикальный вниз.

В качестве критерия выбора очередного пикселя используется минимум модуля разности квадратов расстояний от центра окружности до пикселя и до идеальной окружности. Эти величины имеют следующие значения:

$m_h=|(x_i+1)^2+y_i^2-R^2|$,
$m_d=|(x_i+1)^2+(y_i-1)^2-R^2|$,
$m_v=|x_i^2+(y_i-1)^2-R^2|$,

где m_h, m_d, m_v – искомые величины расстояний для горизонтального, диагонального и вертикального направлений соответственно; x_i, y_i – координаты текущего пикселя.

Вычисления можно упростить, если заметить, что в окрестности текущей точки возможны пять вариантов прохождения окружности (рис. 3):

1) между горизонтальным (x_i+1, y_i) и диагональным (x_i+1, y_i-1) пикселами;

2) между горизонтальным (x_i+1, y_i) и вторым диагональным (x_i+1, y_i+1) пикселами;

3) между вертикальным (x_i, y_i-1) и диагональным (x_i+1, y_i-1);

4) между вертикальным (x_i, y_i-1) и третьим диагональным (x_i-1, y_i-1) пикселами;

5) прохождение точно через диагональный (x_i+1, y_i-1) пиксель.

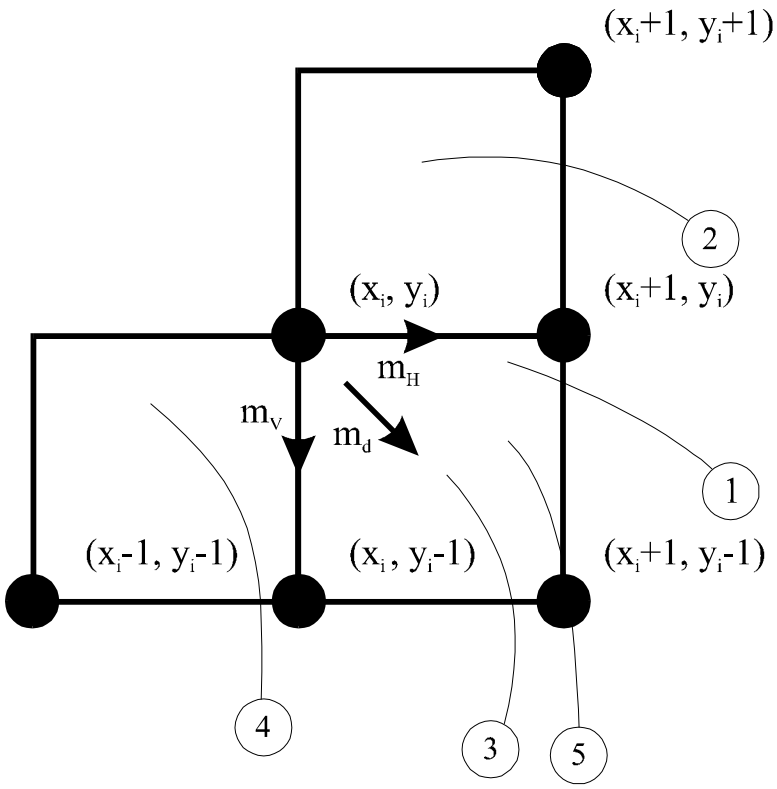


Рис. 3. Пересечение окружности и сетки растра

По аналогии с ошибкой в алгоритме построения отрезков выберем и здесь некоторую величину, анализ знака которой позволит выбрать нужный пиксель.

Такой величиной Δ является разность квадратов расстояний от центра окружности до диагонального (x_i+1,y_i-1) пикселя и до идеальной окружности:

$\Delta_i=(x_i+1)^2+(y_i-1)^2-R^2$.

Отрицательное значение $\Delta_i<0$ может быть только в том случае, когда диагональный пиксель (x_i+1, y_i-1) лежит внутри окружности, т. е. это случаи 1 и 2, и следует выбирать горизонтальный (x_i+1, y_i) или диагональный (x_i+1,y_i-1) пиксель.

Рассмотрим случай 1 и вычислим разность Δ_1 расстояний, т. е.

$\Delta_1=m_h-m_d=|(x_i+1)^2+y_i^2-R^2|-|(x_i+1)^2+(y_i-1)^2-R^2|$.

При $m_h<m_d$ $\Delta_1<0$, и в этом случае расстояние до горизонтального пикселя меньше, чем до диагонального, поэтому выбирается горизонтальный пиксель. При $m_h\geq m_d$ $\Delta_1\geq 0$, и в этом случае расстояние до диагонального пикселя меньше, чем до горизонтального, поэтому выбирается диагональный пиксель. При $m_h=m_d$, $\Delta_1=0$ и можно выбирать либо горизонтальный, либо диагональный пиксель. Для определённости выберем горизонтальный пиксель.

Для сокращения вычислений упростим выражение. При этом следует учесть, что для случая 1 $m_h>0$, а $m_d<0$, так как горизонтальный пиксель лежит вне окружности, а диагональный внутри окружности.

Таким образом, можно записать:

$\Delta_1=(x_i+1)^2+y_i^2-R^2+(x_i+1)^2+(y_i-1)^2-R^2$

Дополнение до полного квадрата члена y_i^2 с помощью добавления и вычитания $-2y_i+1$ даёт

$\Delta_1=2[(x_i+1)^2+(y_i-1)^2-R^2]+2y_i-1.$

В квадратных скобках стоит по определению Δ_i, и его подстановка существенно упрощает выражение

$\Delta_1=2(\Delta_i+y_i)-1.$

Во втором случае выбирается горизонтальный пиксель, второй диагональный пиксель (x_i+1, y_i+1) выбран быть не может в силу монотонного убывания функции. Для этого случая $m_h<0$ и $m_d<0$, для первого квадранта $1<y_i<R$, поэтому $\Delta_1=1-2y_i<0$. Таким образом, и в случае 2, если $\Delta_1<0$, то выбирается горизонтальный пиксель.

Рассмотрим случай $\Delta_i>0$. Это будет справедливо только в том случае, если диагональный пиксель лежит вне окружности, т. е. это случаи 3 и 4, и надо выбирать либо диагональный (x_i+1, y_i-1), либо вертикальный (x_i, y_i-1) пиксель.

Рассмотрим случай 3 и вычислим разность расстояний:

$\Delta_2=m_d-m_v=|(x_i+1)^2+(y_i-1)^2-R^2|-|x_i^2+(y_i-1)^2-R^2|$

При $\Delta_2<0$ расстояние до диагонального пикселя меньше, чем до вертикального, поэтому выбирается диагональный пиксель. При $m_d> m_v$ $\Delta_2>0$ – расстояние до вертикального пикселя меньше, чем до диагонального, поэтому выбирается вертикальный пиксель. При $m_d=m_v$ $\Delta_2=0$, и можно выбирать либо диагональный, либо вертикальный пиксель. Для определённости выберем диагональный пиксель.

Для упрощения вычисления выражения учтём, что для случая 3 $m_d>0$, а $m_v<0$, так как диагональный пиксель лежит вне окружности, а вертикальный внутри окружности:

$\Delta_2=(x_i+1)^2+(y_i-1)^2-R^2+x_i^2+(y_i-1)^2-R^2.$

Дополнение до полного квадрата члена x_i^2 с помощью добавления и вычитания $2x_i+1$ даёт

$\Delta_2=2[(x_i+1)^2+(y_i-1)^2-R^2]-2x_i-1.$

В квадратных скобках стоит по определению Δ_i, и его подстановка существенно упрощает выражение:

$\Delta_2=2(\Delta_i-x_i)-1.$

В четвёртом случае выбирается вертикальный пиксель, третий диагональный пиксель (x_i-1, y_i-1) выбран быть не может в силу монотонного убывания функции. Для этого случая m_d>0 и m_v>0, для первого квадранта 0<x_i<R, поэтому Δ_2=2x_i+1>0. Таким образом, и в случае 4, если Δ_2>0, то выбирается вертикальный пиксель.

Остаётся проверить случай 5, для которого Δ_i=0, то есть диагональный пиксель лежит точно на окружности.

Вычислим значение Δ_1=m_h-m_d, учитывая, что m_h>0, а m_d=0:

$\Delta_1=(x_i+1)^2+y_i^2-R^2>0$,

следовательно, выбирается диагональный пиксель.

Вычислим значение Δ_2=m_d-m_v, учитывая, что m_d=0, m_v<0:

$\Delta_2=x_i^2+(y_i-1)^2-R^2<0$,

следовательно, выбирается диагональный пиксель.

Таким образом, при Δ_i=0 выполняются те же условия для Δ_1 и Δ_2, как и в ранее рассмотренных случаях.

Подведём итог полученных результатов:

Δ_i<0:

$\Delta_1\leqslant 0$ выбираем пиксель (x_i+1, y_i),

Δ_1>0 выбираем пиксель (x_i+1, y_i-1),

Δ_i>0:

$\Delta_2\leqslant 0$ выбираем пиксель (x_i+1, y_i-1),

Δ_2>0 выбираем пиксель (x_i, y_i-1),

Δ_i=0: выбираем пиксель (x_i+1, y_i-1).

Теперь остаётся записать формулы для вычисления координат очередного пикселя и критерия для каждого из трёх возможных направлений движения. Обозначим новое положение пикселя как (i+1) в индексе (x, y) координат и критерия Δ.

При переходе к горизонтальному пикселю:

$x_{i+1}=x_i+1$,

$y_{i+1}=y_i$,

$\Delta_{i+1}=(x_{i+1}+1)^2+(y_{i+1}-1)^2-R^2=(x_{i+1})^2+2x_{i+1}+1+(y_i-1)^2-R^2=$

$=(x_i+1)^2+(y_i-1)^2-R^2+2x_{i+1}+1=\Delta i+2x_{i+1}+1$.

При переходе к диагональному пикселю:

$x_{i+1}=x_i+1$,

$y_{i+1}=y_i-1$,

Δ_{i+1} $\Delta_i+2x_{i+1}-2y_{i+1}+2$.

При переходе к вертикальному пикселю:

$x_{i+1}=x_i$,

$y_{i+1}=y_i-1$,

$\Delta_{i+1}\quad\Delta_i-2y_{i+1}+1$.

Начальное значение для $x=0$, $y=R$ будет равно

$\Delta_i=(0+1)^2+(R-1)^2-R^2=2(1-R)$.

Алгоритм Брезенхема для генерации окружности в первом квадранте может быть записан в следующем виде:

1. Ввод исходных данных R (радиус окружности) и при необходимости x0, y0 (координаты центра окружности).
2. Задание начальных значений текущих координат пикселя x:=0, y:=R, параметров delta:=2-2*R, d1:=0, d2:=0.
3. Проверка окончания работы: если x>y, то переход к п.10.
4. Вывод текущего пикселя (x,y) Здесь же можно вывести остальные семь симметричных.
5. Вычисление d1:=2*(delta +y)-1, d2:=2*(delta -x)-1.
6. Анализ значений параметров:
   - если delta<0 и d1<=0, то переход к п.7;
   - если delta>0 и d2>0, то переход к п.8;
   - в остальных случаях переход к п.9.
7. Вычисление параметра delta и горизонтальный шаг:
   - x:=x+1;
   - delta:=delta+2*x+1;
   - переход к п. 3.
8. Вычисление параметра delta и вертикальный шаг:
   - y:=y-1;
   - delta:=delta-2*y+1;
   - переход к п. 3.
9. Вычисление параметра delta и диагональный шаг:
   - x:=x+1;
   - y:=y-1;
   - delta:=delta +2*(x-y);
   - переход к п. 3.
10. Конец.

Учёт ненулевых координат центра окружности означает фактически выполнение операции переноса, поэтому координаты точек окружности (x,y) будут вычисляться по формуле: (x0+x, y0+y).

Задание

1. Написать программу (на языке высокого уровня), реализующую алгоритмы растеризации окружности с их последующей прорисовкой. Координаты центра и радиуса должны задаваться пользователем. Реализовать:

- построение окружности по уравнению окружности;
- построение окружности по параметрическому уравнению;
- алгоритм Брезенхема для построения окружности;
- встроенные средства языка программирования.

2. Проиллюстрировать разницу между результатами или её отсутствие.

3. Ответить на контрольные вопросы.

4. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Задание.
3. Схемы алгоритмов.
4. Листинг программы.
5. Пример работы программы.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Ознакомление с теоретическими материалами по теме.

4. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

1) чтение файла изображения;
2) растеризацию окружностей в соответствии с заданием по:

– уравнению окружности;

– параметрическому уравнению;
– алгоритму Брезенхема;
– встроенные средства языка программирования;

3) запись полученного изображения в файл.

5. Разработка и отладка программы для решения задачи.
6. Ответы на контрольные вопросы.
7. Оформление отчёта.

Индивидуальные задания

Варианты задания предусматривают вывод нескольких окружностей или их дуг. Для каждой из окружностей нужно в соответствии с условием вычислить значения исходных данных.

1. Вывести вписанную и описанную окружности квадрата по его длине стороны.
2. Вывести вписанную и описанную окружности по заданной стороне правильного пятиугольника.
3. Вывести вписанную и описанную окружности по заданной стороне правильного шестиугольника.
4. Вывести арбеллос.
5. Вывести салинон.
6. Вывести трикветр.
7. Вывести цветок жизни.
8. Вывести треугольник Рёло заданной ширины.
9. Вывести вписанную и описанную окружности по заданной стороне правильного семиугольника.
10. Вывести вписанную и описанную окружности по заданной стороне правильного восьмиугольника.
11. По заданным вершинам вывести треугольник и его вписанную и описанную окружности.
12. По заданным длинам сторон вывести треугольник и его вневписанные окружности.
13. По двум заданным радиусам вывести яйцо Мосса.
14. По двум заданным осям вывести стадиум.
15. По заданным сторонам треугольника вывести окружности Мальфатти.
16. По радиусу и длине оси вывести овал.
17. По заданным радиусам кривизны и толщине нарисовать линзу.

18. По заданным радиусам и расстоянию между центрами нарисовать луночку.
19. Вывести трифолий.
20. Вывести Инь-Ян.

Контрольные вопросы

1. Что такое пиксель?
2. Что такое растровое изображение?
3. Для чего нужны алгоритмы растеризации окружностей?
4. Какие ещё алгоритмы растеризации окружностей существуют?

Пример выполнения работы

0 По двум заданным длинами осям вывести овал.

Для растеризации овала, заданного длинами осей, реализуем алгоритмы растеризации дуг окружностей по уравнению окружности, параметрическому уравнению и Брезенхема в виде отдельных функций, куда будем передавать координаты центра, радиусы, координаты лучей, определяющих начало и конец дуги и холст.

На форме (рис. 4) предусматриваем 2 объекта Tbutton для нового изображения и сохранения результата, диалог TsavePictureDialog для цивилизованного сохранения файла, объект TImage для хранения и отображения картинок, 2 объекта Tedit для ввода длин осей, 2 компонента Tlabel для подписи Tedit.

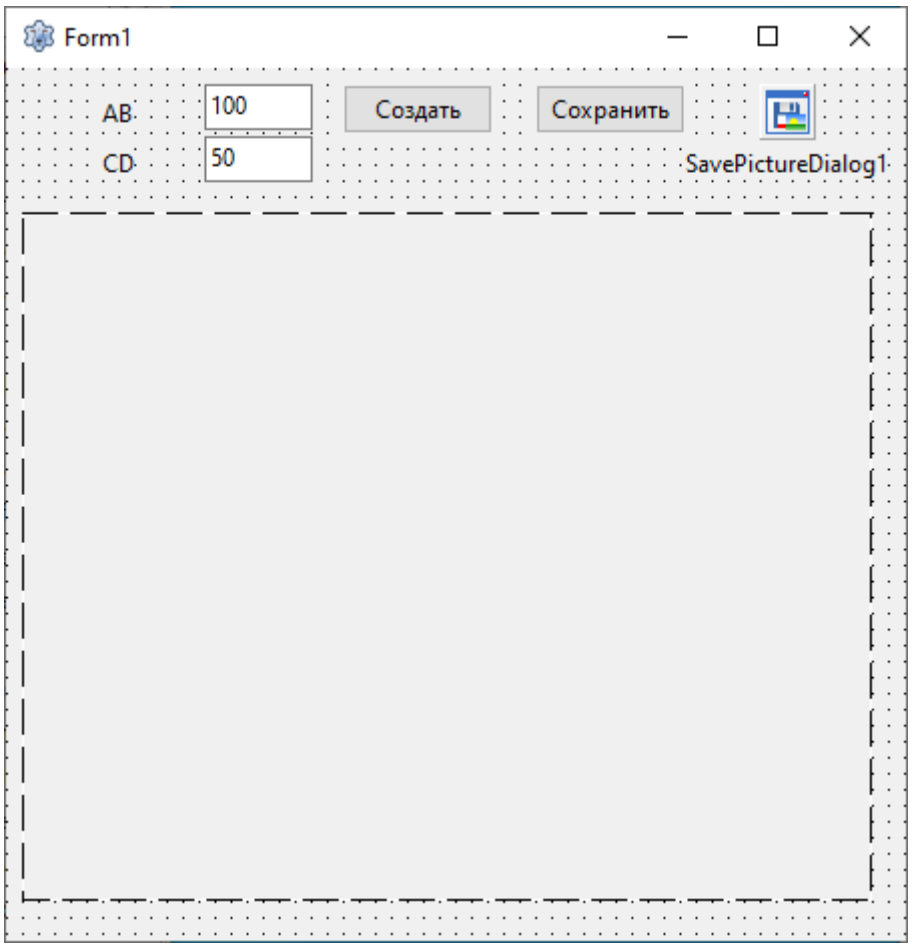


Рис. 4. Форма программы прорисовки окружностей

При нажатии на кнопку «Создать» из компонентов Tedit считываются длины осей, для битмапа Image1 выполняется очистка, задаётся формат пикселей pf24bit, задаются ширина и высота по размерам длин осей + 50, чтобы поместился овал, обрамлённый пустотой, выполняется заполнение белым фоном.

Далее вычисляются параметры дуг окружностей и разным цветом прорисовываются дуги по уравнению окружности, параметрическому уравнению, алгоритму Брезенхема и встроенным методом Tcanvas.arc(). Методика построения четырёхцентрового овала по заданным длинам осей приведена в [5]. Примеры построения дуг окружностей приведены в [6].

Получили модуль приблизительно следующего содержания:

```
unit Unit1;

{$mode objfpc}{$H+}
```

```
interface

uses
  Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs,
StdCtrls,
  GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,
ExtDlgs,
  fpImage,  crt, math;
type

  { TForm1 }

  TForm1 = class(TForm)
    Button1: TButton;
    Button3: TButton;
    Edit6: TEdit;
    Edit7: TEdit;
    Label1: TLabel;
    Label3: TLabel;
    SavePictureDialog1: TSavePictureDialog;
    Image1: TImage;
    procedure Button1Click(Sender: TObject);
    procedure Button3Click(Sender: TObject);
  private
    { private declarations }
  public
    { public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.lfm}

{ TForm1 }
```

```
procedure arceq(x0, y0, r, x1,y1,x2,y2:real; canvas:TCanvas);
var x,y,kp,bp:real;
  inv:integer;
begin
  if x1>=x2 then inv:=-1 else inv:=1;//нужна ли инверсия
  if x1=x2 then x1:=x1+0.1/r;// плутовство исключить вертикали.
  kp:=(y1-y2)/(x1-x2);//угловой коэффициент секущей хорды
  bp:=y1-x1*kp;//свободный член секущей хорды
  //прорисовываем как функцию y=f(x) по точкам
  x:=0;// начиная с x=0
  y:=r;
  while x<y do //до угла 45
    begin //если точки с нужной стороны секущей
      y:=sqrt(sqr(r)-sqr(x));//вывод точек в симметричных секторах
      if inv*((y0+y)-(kp*(x0+x)+bp))<=0 then
           canvas.Pixels[floor(x0+x),floor(y0+y)]:=canvas.Pen.Color;
      if inv*((y0-y)-(kp*(x0+x)+bp))<=0 then
           canvas.Pixels[floor(x0+x),floor(y0-y)]:=canvas.Pen.Color;
      if inv*((y0+y)-(kp*(x0-x)+bp))<=0 then
           canvas.Pixels[floor(x0-x),floor(y0+y)]:=canvas.Pen.Color;
      if inv*((y0-y)-(kp*(x0-x)+bp))<=0 then
           canvas.Pixels[floor(x0-x),floor(y0-y)]:=canvas.Pen.Color;
      x:=x+1
    end;
  while y>=0 do //с угла 45 до y=0 выводим ка функцию x=g(y)
    begin //если точки с нужной стороны секущей
      x:=sqrt(sqr(r)-sqr(y)); //вывод точек в симметричных секторах
      if inv*((y0+y)-(kp*(x0+x)+bp))<=0 then
           canvas.Pixels[floor(x0+x),floor(y0+y)]:=canvas.Pen.Color;
      if inv*((y0-y)-(kp*(x0+x)+bp))<=0 then
           canvas.Pixels[floor(x0+x),floor(y0-y)]:=canvas.Pen.Color;
      if inv*((y0+y)-(kp*(x0-x)+bp))<=0 then
           canvas.Pixels[floor(x0-x),floor(y0+y)]:=canvas.Pen.Color;
      if inv*((y0-y)-(kp*(x0-x)+bp))<=0 then
           canvas.Pixels[floor(x0-x),floor(y0-y)]:=canvas.Pen.Color;
      y:=y-1
```

```
    end;
end;


procedure arcparam(x0, y0, r, x1,y1,x2,y2:real; canvas:TCanvas);
var x,y,tau,tau0,sa,ea:real;
begin
  if x1=x0 then if y1>y0 then sa:=pi/2 else sa:=3*pi/2
    else sa:=arctan((y1-y0)/(x1-x0));//начальный угол
  if x2=x0 then if y2>y0 then ea:=pi/2 else ea:=3*pi/2
    else ea:=arctan((y2-y0)/(x2-x0));//конечный угол
  if x1<x0 then sa:=pi+sa;//коррекция полуплоскости нач угла
  if x2<x0 then ea:=pi+ea;//коррекция полуплоскости кон угла
  if sa<0 then sa:=sa+2*pi;//коррекция знака начального угла
  if ea<0 then ea:=ea+2*pi;//коррекция знака конечного угла
  if sa>ea then//если начальный угол больше конечного
      ea:=ea+2*pi;//доворачиваем конечный угол на оборот
  tau0:=1/r;//чем больше радиус тем меньше шаг.
  tau:=sa;//начинаем с начального угла
  while tau<ea do //рисуем до конечного
      begin
        x:=x0+r*cos(tau);// параметрическое уравнение для x
        y:=y0+r*sin(tau);// параметрическое уравнение для y
        canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color;// точка
        tau:=tau+tau0;//увеличиваем угол
      end;
end;


procedure arcbrez(x0, y0, r, x1,y1,x2,y2:integer; canvas:TCanvas);
const pi=3.1415926;
var x,y,delta,spx,epx,spy,epy,sag,eag,sp,ep:integer;
  sa,ea,rs,re,t:real;
  dr,inv:boolean;
begin
  x:=0; //начальное значение x для алгоритма Брезенхема
  y:=R; //начальное значение y для алгоритма Брезенхэма
  delta:=2-2*r; //ошибка
```

```
rs:=sqrt(sqr(x1-x0)+sqr(y1-y0));//длина луча начального угла
re:=sqrt(sqr(x2-x0)+sqr(y2-y0));//длина луча конечного угла
sa:=arcsin((y1-y0)/rs);//начальный угол
ea:=arcsin((y2-y0)/re);//конечный угол
if x1<x0 then sa:=pi-sa;//коррекция полуплоскости нач угла
if x2<x0 then ea:=pi-ea;//коррекция полуплоскости кон угла
if sa<0 then sa:=sa+2*pi;//коррекция знака начального угла
if ea<0 then ea:=ea+2*pi;//коррекция знака конечного угла
inv:=false;//нужна ли инверсия
if sa>ea then//если начальный угол больше конечного
  begin
    t:=sa;
    sa:=ea;
    ea:=t; //меняем их местами
    inv:=true;//и инвертируем вывод
  end;
sag:=round(sa*360/2/pi);//начальный угол в градусах
eag:=round(ea*360/2/pi);//конечный угол в градусах
spx:=round(x0+r*cos(sa));//абсцисса начала сектора
spy:=round(y0+r*sin(sa));//ордината начала сектора
epx:=round(x0+r*cos(ea));//абсцисса конца сектора
epy:=round(y0+r*sin(ea));//ордината конца сектора
while x<=y do //вывод по алгоритму Бразенхейма 8 секторов
  begin
      dr:=false;//если есть все условия вывода для каждого сектора
      if (sag<=45)and(eag>=0) then
        begin
          if (sag>=0) then sp:=spy else sp:=y0;
          if (eag<45) then ep:=epy else ep:=y0+r;
          if (y0+x>=sp) and (y0+x<=ep) then dr:=true;
          end;
        if (dr and (not inv)) or (inv and (not dr)) then
            canvas.Pixels[x0+y,y0+x]:=canvas.Pen.Color;//1 (обр)
      dr:=false;
      if (sag<90) and (eag>=45) then
        begin
          if (sag>=45) then sp:=spx else sp:=x0+r;
          if (eag<90) then ep:=epx else ep:=x0;
```

```
    if (x0+x <= sp) and (x0+x >= ep) then dr:=true;
  end;
if (dr and (not inv)) or (inv and (not dr)) then
      canvas.Pixels[x0+x,y0+y]:=canvas.Pen.Color;//2
dr:=false;
if (sag<135) and (eag>=90) then
  begin
    if (sag>=90) then sp:=spx else sp:=x0;
    if (eag<135) then ep:=epx else ep:=x0-r;
    if (x0-x <= sp) and (x0-x >= ep) then dr:=true;
  end;
if (dr and (not inv)) or (inv and (not dr)) then
      canvas.Pixels[x0-x,y0+y]:=canvas.Pen.Color;//3  (обр)
dr:=false;
if (sag<180) and (eag>=135) then
  begin
    if (sag>=135) then sp:=spy else sp:=y0+r;
    if (eag<180) then ep:=epy else ep:=y0;
    if (y0+x <= sp) and (y0+x >= ep) then dr:=true;
  end;
if (dr and (not inv)) or (inv and (not dr)) then
      canvas.Pixels[x0-y,y0+x]:=canvas.Pen.Color;//4
dr:=false;
if (sag<225) and (eag>=180) then
  begin
    if (sag>=180) then sp:=spy else sp:=y0;
    if (eag<225) then ep:=epy else ep:=y0-r;
    if (y0-x <= sp) and (y0-x >= ep) then dr:=true;
  end;
if (dr and (not inv)) or (inv and (not dr)) then
      canvas.Pixels[x0-y,y0-x]:=canvas.Pen.Color;//5  (обр)
dr:=false;
if (sag<270) and (eag>=225) then
  begin
    if (sag>=225) then sp:=spx else sp:=x0-r;
    if (eag<270) then ep:=epx else ep:=x0;
    if (x0-x >= sp) and (x0-x <= ep) then dr:=true;
  end;
```

```
        if (dr and (not inv)) or (inv and (not dr)) then
            canvas.Pixels[x0-x,y0-y]:=canvas.Pen.Color;//6
        dr:=false;
        if (sag<315) and (eag>=270) then
          begin
            if (sag>=270) then sp:=spx else sp:=x0;
            if (eag<315) then ep:=epx else ep:=x0+r;
            if (x0+x >= sp) and (x0+x <= ep) then dr:=true;
          end;
        if (dr and (not inv)) or (inv and (not dr)) then
            canvas.Pixels[x0+x,y0-y]:=canvas.Pen.Color;//7 (обр)
        dr:=false;
        if (sag<360) and (eag>=315) then
          begin
            if (sag>=315) then sp:=spy else sp:=y0-r;
            if (eag<360) then ep:=epy else ep:=y0;
            if (y0-x >= sp) and (y0-x <= ep) then dr:=true;
          end;
        if (dr and (not inv)) or (inv and (not dr)) then
            canvas.Pixels[x0+y,y0-x]:=canvas.Pen.Color;//8
//      Переход к следующей точке
      if (delta<0) and (2*(delta+y)-1<=0) then
          begin
            x:=x+1;
            delta:=delta+2*x+1
          end
      else
        if (delta>0) and (2*(delta-x)-1>0) then
            begin
              y:=y-1;
              delta:=delta-2*y+1
            end
        else
          begin
              x:=x+1;
              y:=y-1;
              delta:=delta+2*(x-y)
          end;
```

```
    end;
end;


procedure TForm1.Button1Click(Sender: Tobject); //Кнопа Создать1
var ab,cd,bord:integer;
    oa,oc,xa,ya,xb,yb,xc,yc,xd,yd,xo,yo:real;
    ce,ca,ae,xe,ye,xf,yf,kac,bac:real;
    kgl,bgl,xk,yk,xl,yl:real;
    r1,r2,xm,ym,xn,yn,sxg,xg,yg,xh,yh,xi,yi,xj,yj:real;
begin
  with Image1.Picture.Bitmap do //С битмапом на форме TImage
    begin
      Clear; //а вдруг там мусор
      PixelFormat:=pf24bit; //формат пикселей
      AB:=StrToInt(Edit6.Text);//большая ось овала
      CD:=StrToInt(Edit7.Text);//малая ось овала
      bord:=50;//бордюр
      Width:=AB + 2*bord; //Ширина
      Height:=CD + 2*bord; //Высота
      Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон
      canvas.Pen.Color:=RGB(127,127,127); //цвет точек
      OA:=AB/2;//половина большой оси
      OC:=CD/2;//половина малой оси
      xa:=0;//абсцисса точки A
      ya:=OC;//ордината точки A
      xb:=xa+ab;//абсцисса точки B
      yb:=ya;//ордината точки B
      xc:=OA;//абсцисса точки C
      yc:=0;//ордината точки C
      xd:=xc;//абсцисса точки D
      yd:=yc+cd;//ордината точки D
      xo:=xc;//абсцисса точки O
      yo:=ya;//ордината точки O
      CA:=sqrt(sqr(OA)+sqr(OC));//длина отрезка CA
      CE:=OA-OC;//длина отрезка CE
      AE:=CA-CE;//Длина отрезка AE
      xe:=(xa*CE+xc*AE)/CA;//абсцисса точки E
```

```
ye:=(ya*CE+yc*AE)/CA;//ордината точки E
xf:=(xe+xa)/2;//абсцисса точки F, AF=FE
yf:=(ye+ya)/2;//ордината точки F
kac:=-oc/oa;//угловой коэффициент прямой AC
bac:=ya-xa*kac;//свободный член прямой AC
kgl:=-1/kac;//угловой коэффициент прямой GL
bgl:=xf/kac+yf;//свободный член прямой GL
```

//центры малых окружностей овала – в точке K на пересечении AB и GL и в точке M симметрично.

```
yk:=ya; //ордината точки K
xk:=(yk-bgl)/kgl; //абсцисса точки K
r1:=xk-xa; //радиус малой окружности
xm:=xb-r1; //абсцисса точки M
ym:=yb; //ордината точки M
```

//центр большой окружности овала - в точке L на пересечении CD и GL и в точке N симметрично.

```
xl:=xc; //абсцисса точки L
yl:=kgl*xl+bgl; //ордината точки L
r2:=yl-yc; //радиус большой окружности
xn:=xd; //абсцисса точки N
yn:=yo-(yl-yo); //ордината точки N
//Координаты точки G сопряжения овала
sxg:=-sqr(yk)+(2*kgl*xk+2*bgl)*yk-sqr(kgl)*sqr(xk)-
2*bgl*kgl*xk+(sqr(kgl)+1)*sqr(r1)-sqr(bgl);
xg:=min((sqrt(sxg)+kgl*yk+xk-bgl*kgl)/(sqr(kgl)+1),(-
sqrt(sxg)+kgl*yk+xk-bgl*kgl)/(sqr(kgl)+1));
yg:=kgl*xg+bgl; //ордината точки G
// точки сопряжения H, J, и I находятся симметрично.
xh:=xg; //абсцисса точки H
yh:=yk+(yk-yg); //ордината точки H
xi:=xl+(xl-xg); //абсцисса точки I
yi:=yg; //ордината точки I
xj:=xi; //абсцисса точки J
yj:=yh; //ордината точки J
canvas.Pen.Color:=RGB(191,191,191); //вывод осей серым
can-
vas.line(round(bord+xa),round(bord+ya),round(bord+xb),round(bord+yb))
;
```

```
can-
vas.line(round(bord+xc),round(bord+yc),round(bord+xd),round(bord+yd))
;
canvas.Pen.Color:=RGB(191,191,255);
//вывод овала по уравнениям окружностей - голубым
arceq(bord+xk,bord+yk,r1,bord+xh,bord+yh,bord+xg,bord+yg,canvas);
arceq(bord+xm,bord+ym,r1,bord+xi,bord+yi,bord+xj,bord+yj,canvas);
arceq(bord+xl,bord+yl,r2,bord+xg,bord+yg,bord+xi,bord+yi,canvas);
arceq(bord+xn,bord+yn,r2,bord+xj,bord+yj,bord+xh,bord+yh,canvas);
canvas.Pen.Color:=RGB(255,191,191);
 //вывод овала по параметрическому уравнению - розовым
arcparam(bord+xk,bord+yk,r1,bord+xh,bord+yh,bord+xg,bord+yg,can-
vas);
arcparam(bord+xm,bord+ym,r1,bord+xi,bord+yi,bord+xj,bord+yj,canvas);
arcparam(bord+xl,bord+yl,r2,bord+xg,bord+yg,bord+xi,bord+yi,canvas);
arcparam(bord+xn,bord+yn,r2,bord+xj,bord+yj,bord+xh,bord+yh,canvas);
        canvas.Pen.Color:=RGB(191,255,191);
        //вывод овала по алгоритму Брезенхайма - зелёненьким
arcbrez(round(bord+xk),round(bord+yk),round(r1),
round(bord+xh),round(bord+yh),round(bord+xg),round(bord+yg),canvas);
arcbrez(round(bord+xm),round(bord+ym),round(r1),
round(bord+xi),round(bord+yi),round(bord+xj),round(bord+yj),canvas);
arcbrez(round(bord+xl),round(bord+yl),round(r2),
round(bord+xg),round(bord+yg),round(bord+xi),round(bord+yi),canvas);
arcbrez(round(bord+xn),round(bord+yn),round(r2),
round(bord+xj),round(bord+yj),round(bord+xh),round(bord+yh),canvas);
        canvas.Pen.Color:=RGB(191,191,191);
        //вывод овала встроенными средствами - серым.
canvas.Arc(round(bord+xa),round(bord+ya-
r1),round(bord+xa+2*r1),round(bord+ya+r1),
round(bord+xg),round(bord+yg),round(bord+xh),round(bord+yh));
canvas.Arc(round(bord+xb-2*r1),round(bord+yb-
r1),round(bord+xb),round(bord+yb+r1),
round(bord+xj),round(bord+yj),round(bord+xi),round(bord+yi));
canvas.Arc(round(bord+xc-
r2),round(bord+yc),round(bord+xc+r2),round(bord+yc+2*r2),
round(bord+xi),round(bord+yi),round(bord+xg),round(bord+yg));
```

```
canvas.Arc(round(bord+xd-r2),round(bord+yd-
2*r2),round(bord+xd+r2),round(bord+yd),
round(bord+xh),round(bord+yh),round(bord+xj),round(bord+yj));
        end;
    end;


    procedure TForm1.Button3Click(Sender: TObject);
    begin
      if SavePictureDialog1.Execute then
          Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
    end;

    end.
```

Получен результат (рис. 5):

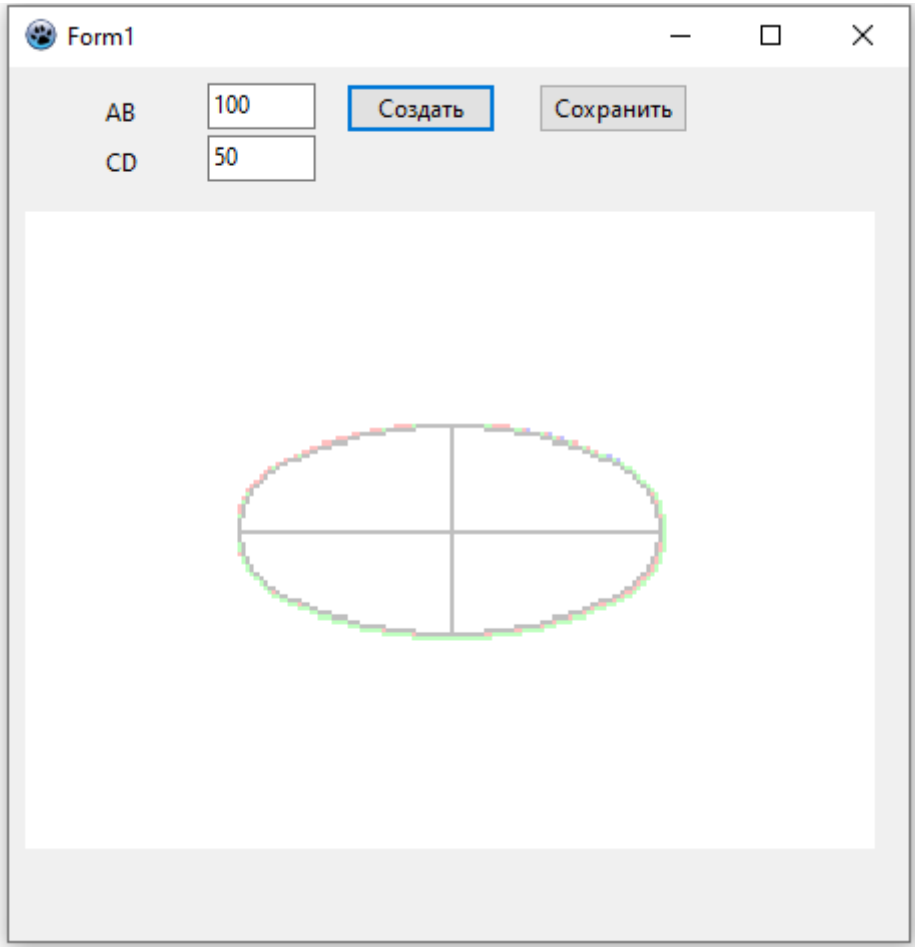


Рис. 5. Результат работы программы прорисовки окружностей